

## 6. Dependency Inversion Principle (DIP) in Java

### 6.1 Introduction

The Dependency Inversion Principle (DIP) is the fifth and final SOLID principle. It encourages decoupling by ensuring that:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

### 6.2 Problem It Solves

Without DIP, high-level classes (business logic) directly depend on low-level implementations (e.g., database, filesystem, notification). This tight coupling makes the code hard to test and change.

### 6.3 Definition of DIP

"Depend on abstractions, not on concretions."

### 6.4 Real-World Analogy

If your TV remote works with multiple TV brands, it must depend on an interface (like IR protocol), not on a specific brand's implementation.

### 6.5 Symptoms of DIP Violation

- Business logic classes use new keyword heavily
- Hard-to-test or mock classes
- Replacing a dependency requires touching multiple files

### 6.6 Benefits of DIP

- Decouples high-level logic from low-level details
- Enhances testability and maintainability
- Promotes modular, pluggable systems

### 6.7 Implementation Guidelines

- Use interfaces or abstract classes for dependencies
- Inject dependencies via constructors or setters
- Use IoC containers (e.g., Spring) for managing dependencies

### 6.8 Examples

#### Example1: Violation: Monitor Coupling

```
package com.mannemlokesh.violation.example1;

class Keyboard {
    public void input() {
        System.out.println("Typing input...");
    }
}
```

```
class Computer {
    private Keyboard keyboard = new Keyboard(); // tightly coupled

    public void use() {
        keyboard.input();
    }
}

public class Client {
    public static void main(String[] args) {
        Computer pc = new Computer();
        pc.use();
    }
}
```

### Example1: Compliant : Monitor Coupling

```
package com.mannemlokesh.compliant.example1;

interface InputDevice {
    void input();
}

class Keyboard implements InputDevice {
    public void input() {
        System.out.println("Typing input...");
    }
}

class Computer {
    private InputDevice device;

    public Computer(InputDevice device) {
        this.device = device;
    }

    public void use() {
        device.input();
    }
}

public class ClientFixed {
    public static void main(String[] args) {
        InputDevice keyboard = new Keyboard();
        Computer pc = new Computer(keyboard);
        pc.use();
    }
}
```

### Output:

```
Typing input...
```

### Example2: Violation: Notification Service

```
package com.mannemlokesh.violation.example2;
```

```
class EmailService {
    public void send(String msg) {
        System.out.println("Sending Email: " + msg);
    }
}

class NotificationManager {
    private EmailService emailService = new EmailService();

    public void notify(String msg) {
        emailService.send(msg);
    }
}
```

#### Example2: Compliant : Notification Service

```
package com.mannemlokes.compliant.example2;

interface MessageService {
    void send(String msg);
}

class EmailService implements MessageService {
    public void send(String msg) {
        System.out.println("Sending Email: " + msg);
    }
}

class NotificationManager {
    private MessageService service;

    public NotificationManager(MessageService service) {
        this.service = service;
    }

    public void notify(String msg) {
        service.send(msg);
    }
}

public class NotificationClient {
    public static void main(String[] args) {
        MessageService email = new EmailService();
        NotificationManager manager = new NotificationManager(email);
        manager.notify("Welcome to DIP");
    }
}
```

#### Output:

```
Sending Email: Welcome to DIP
```

#### Example3: Complaint : Payment Service with DIP

```
package com.mannemlokes.compliant.example3;
```

```

interface PaymentGateway {
    void pay(double amount);
}

class RazorPay implements PaymentGateway {
    public void pay(double amount) {
        System.out.println("Paid using RazorPay: " + amount);
    }
}

class Stripe implements PaymentGateway {
    public void pay(double amount) {
        System.out.println("Paid using Stripe: " + amount);
    }
}

class PaymentService {
    private PaymentGateway gateway;

    public PaymentService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public void processPayment(double amount) {
        gateway.pay(amount);
    }
}

public class PaymentClient {
    public static void main(String[] args) {
        PaymentGateway gateway = new RazorPay();
        PaymentService service = new PaymentService(gateway);
        service.processPayment(1000);
    }
}

```

**Output:**

```
Paid using RazorPay: 1000.0
```

## 6.9 Common Pitfalls

- Confusing DIP with Dependency Injection (DIP is the principle; DI is the technique)
- Interface bloating
- Over-abstraction without real benefit

## 6.10 DIP vs SRP vs OCP

| Principle | Focus                                      |
|-----------|--------------------------------------------|
| DIP       | High-level code depends on abstractions    |
| SRP       | One responsibility per class               |
| OCP       | Extend behavior without modifying existing |

## 6.11 When to Apply DIP

- When high-level modules depend on low-level implementations

- When unit testing becomes difficult
- When building plugins or services

## 6.12 Summary

| Feature     | Description                             |
|-------------|-----------------------------------------|
| Intent      | Invert dependency on concrete classes   |
| Use Case    | Service communication, I/O, integration |
| Key Benefit | Decoupled, flexible, and testable code  |

The Dependency Inversion Principle encourages flexibility and testability by reducing direct dependency on low-level details.